

Web Systems Application Development

Jason Kuruzovich

kuruzj@rpi.edu



Agenda

- Survey Results / Group Setup Coming on Friday

Reading

LEARN THE CORE

- 1 Docker foundations & architecture**
Chs. 1–2
- 2 Installing Docker**
Ch. 3
- 3 Working With Docker Images**
Ch. 4
- 4 Working With Docker Containers**

APPLY & EXTEND

- 6 Exploring Docker**
Ch. 6

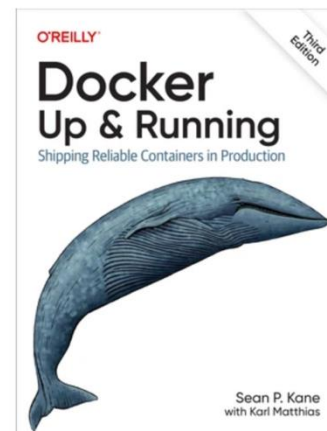
- Available [through the library](#)
- **Read chapters 1, 2, 4, 5, and 6**
— stop at the end of **getting inside a running container**.

BOOK

Docker: Up & Running, 3rd Edition

★★★★★ [1 review](#)

By [Sean P. Kane](#), [Karl Matthias](#)



TIME TO COMPLETE:
11h 17m

LEVEL:
Intermediate to advanced

SKILLS:
[Docker](#)

PUBLISHED BY:
[O'Reilly Media, Inc.](#)

PUBLICATION DATE:
April 2023

PRINT LENGTH:
418 pages

[+ Add to playlist](#)

Start

ITWS 2110 · WEB SYSTEMS DEVELOPMENT · FALL 2026

Where You're Starting From

What the day-one survey says about this class

48

of 60 responded

12

skills rated

1-5

confidence scale

What the numbers mean

1

Never heard of it

2

Heard of it, couldn't do it

3

Could do it with docs open

4

Could do it from memory

5

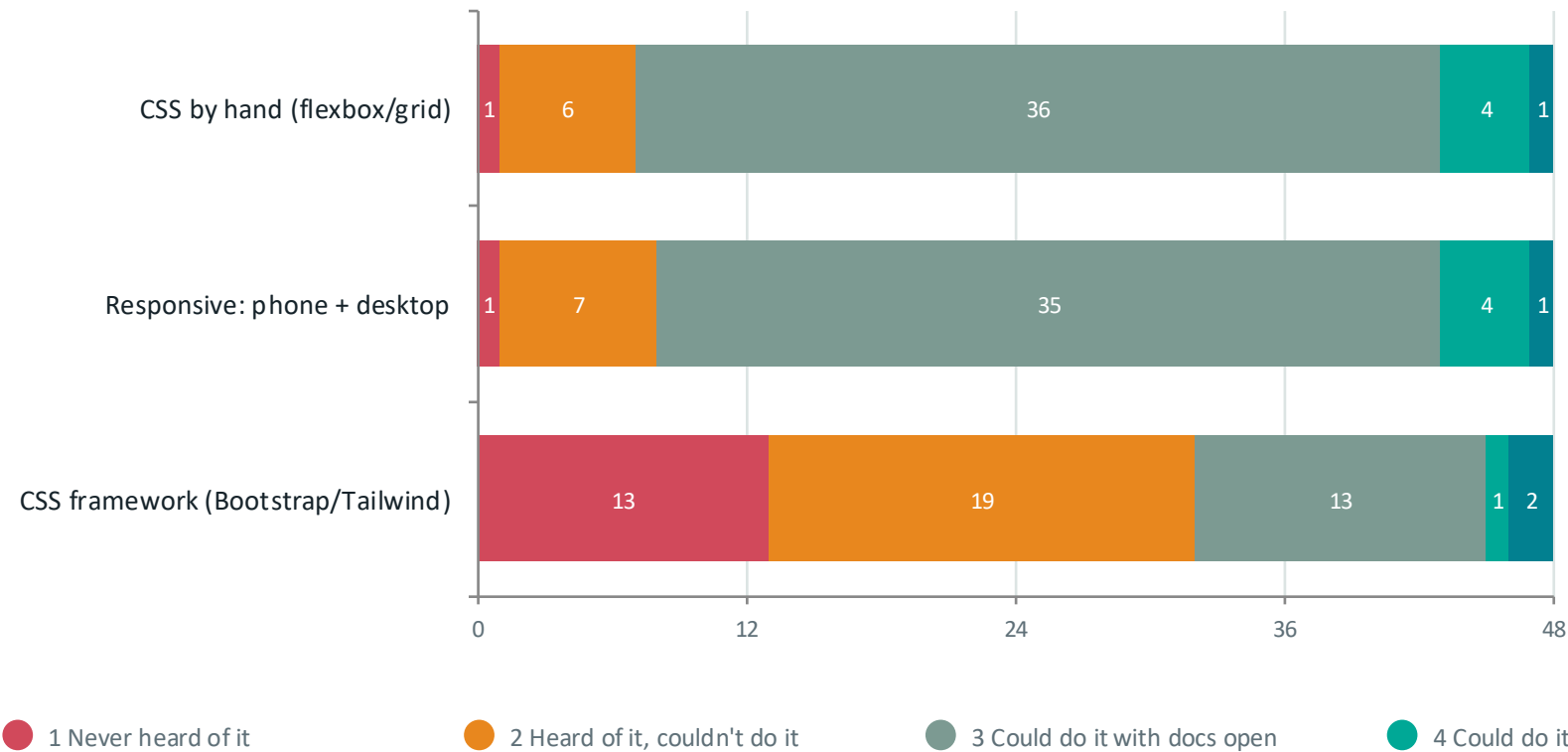
Could teach it

80%

of all 576 answers landed at 3 or below — “could do it with docs open” or less.

This class is unusually uniform. Almost nobody is far ahead, and almost nobody is lost.

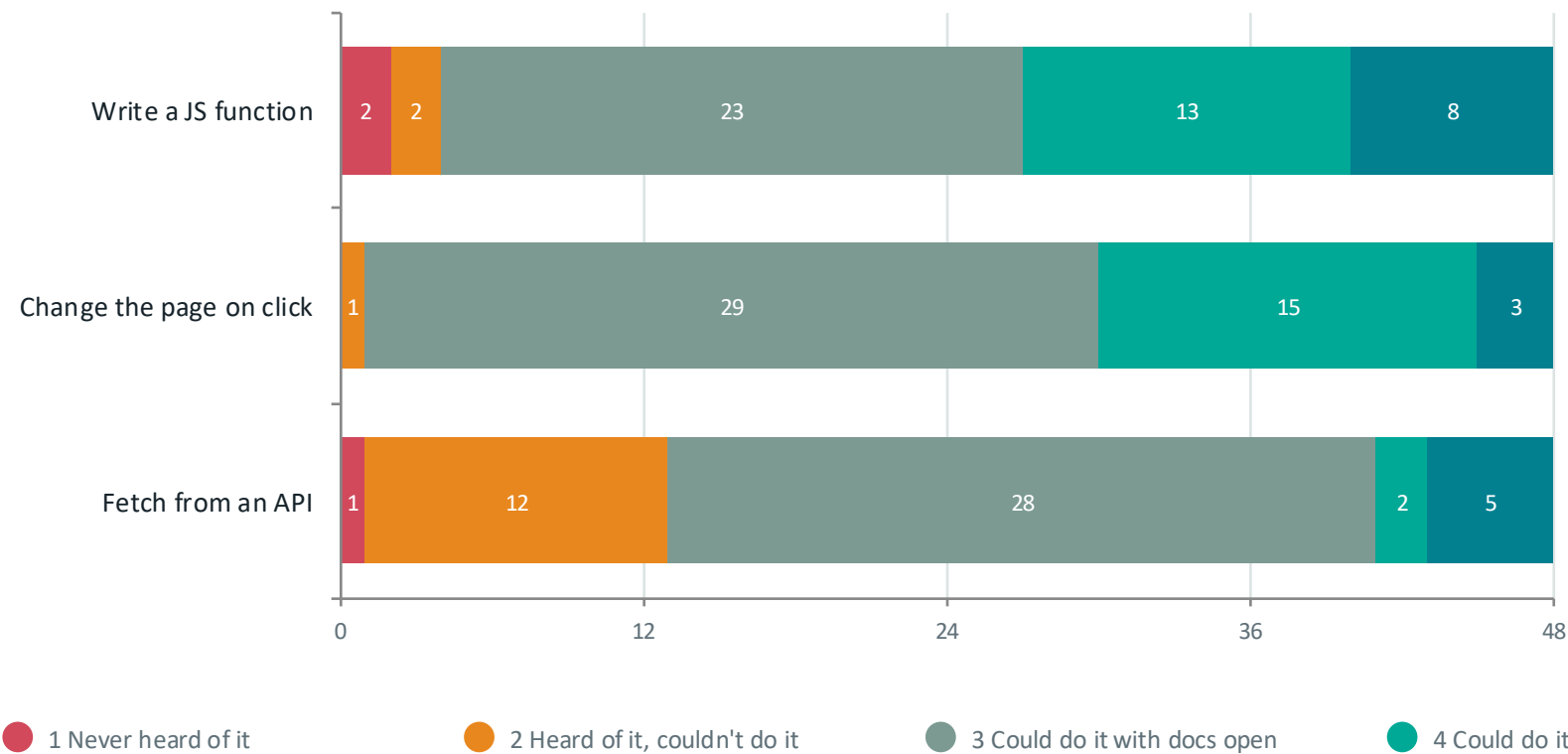
CSS



2.69

group average, 1–5

JavaScript



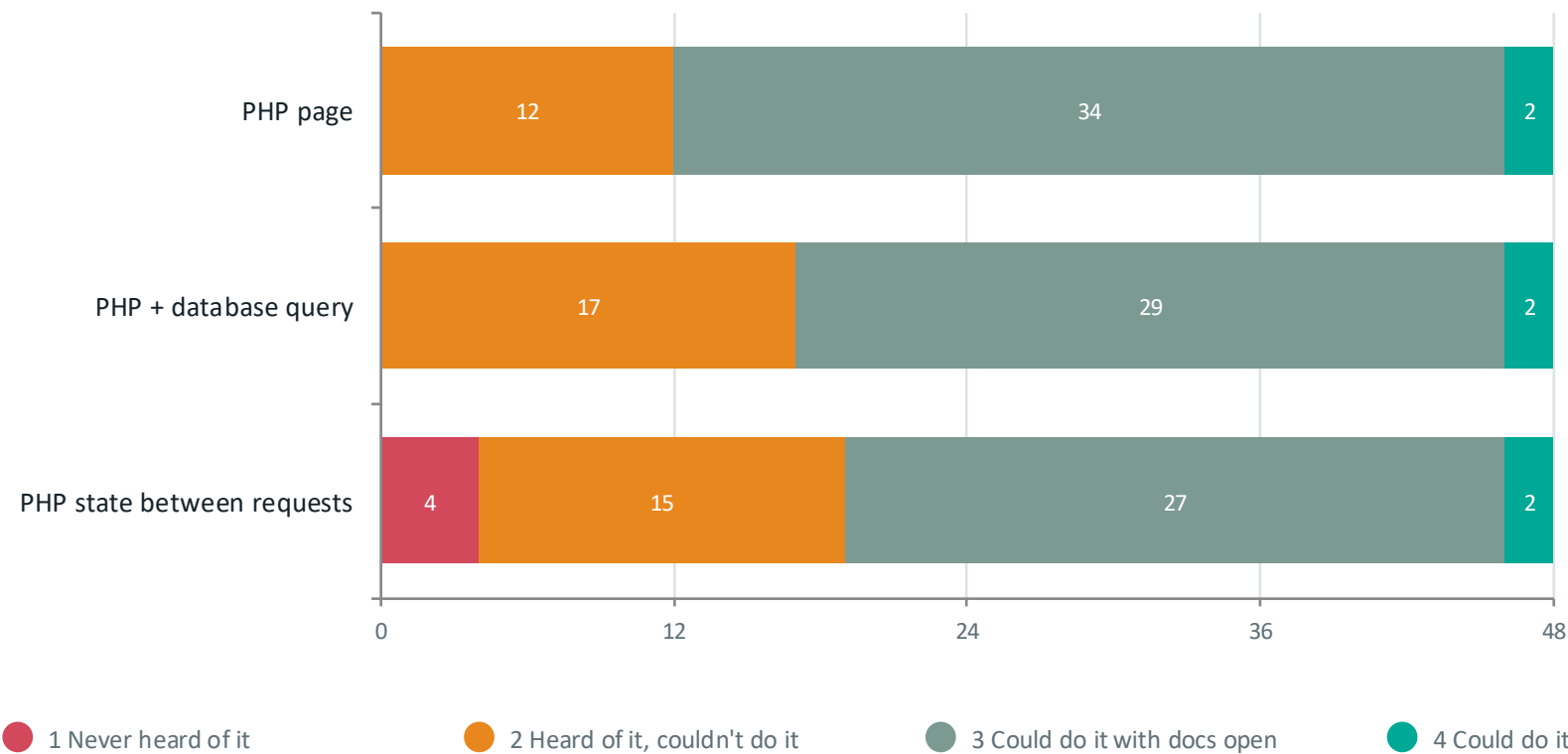
3.29

group average, 1–5

Your strongest area. Writing a function and reacting to a click are close to comfortable.

Fetching from an API is where it drops off.

PHP



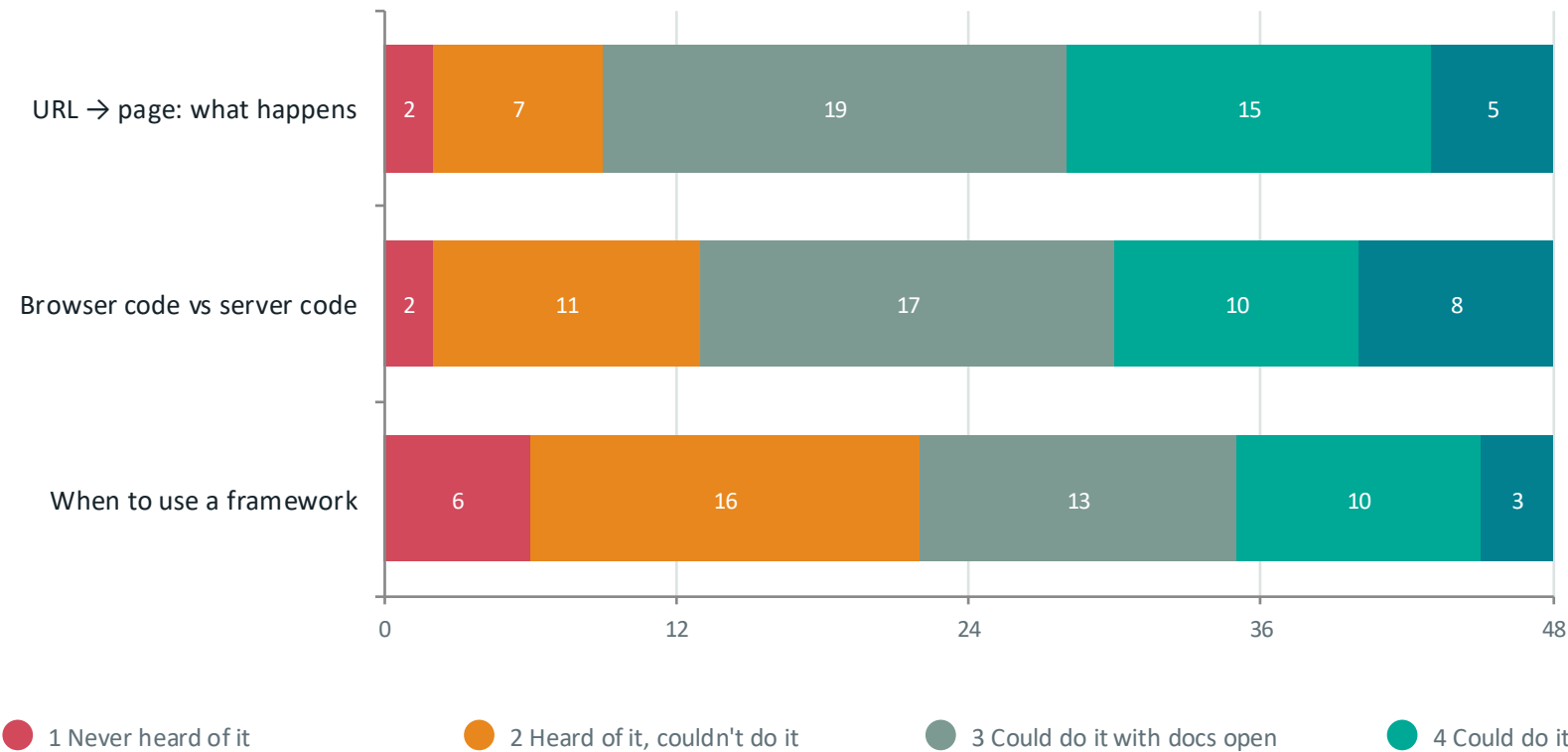
2.68

group average, 1–5

The flattest result here: 96% answered 3 or below on every PHP question, and nobody chose 5.

Homework 1 assumes exactly this.

How systems fit together



3.09

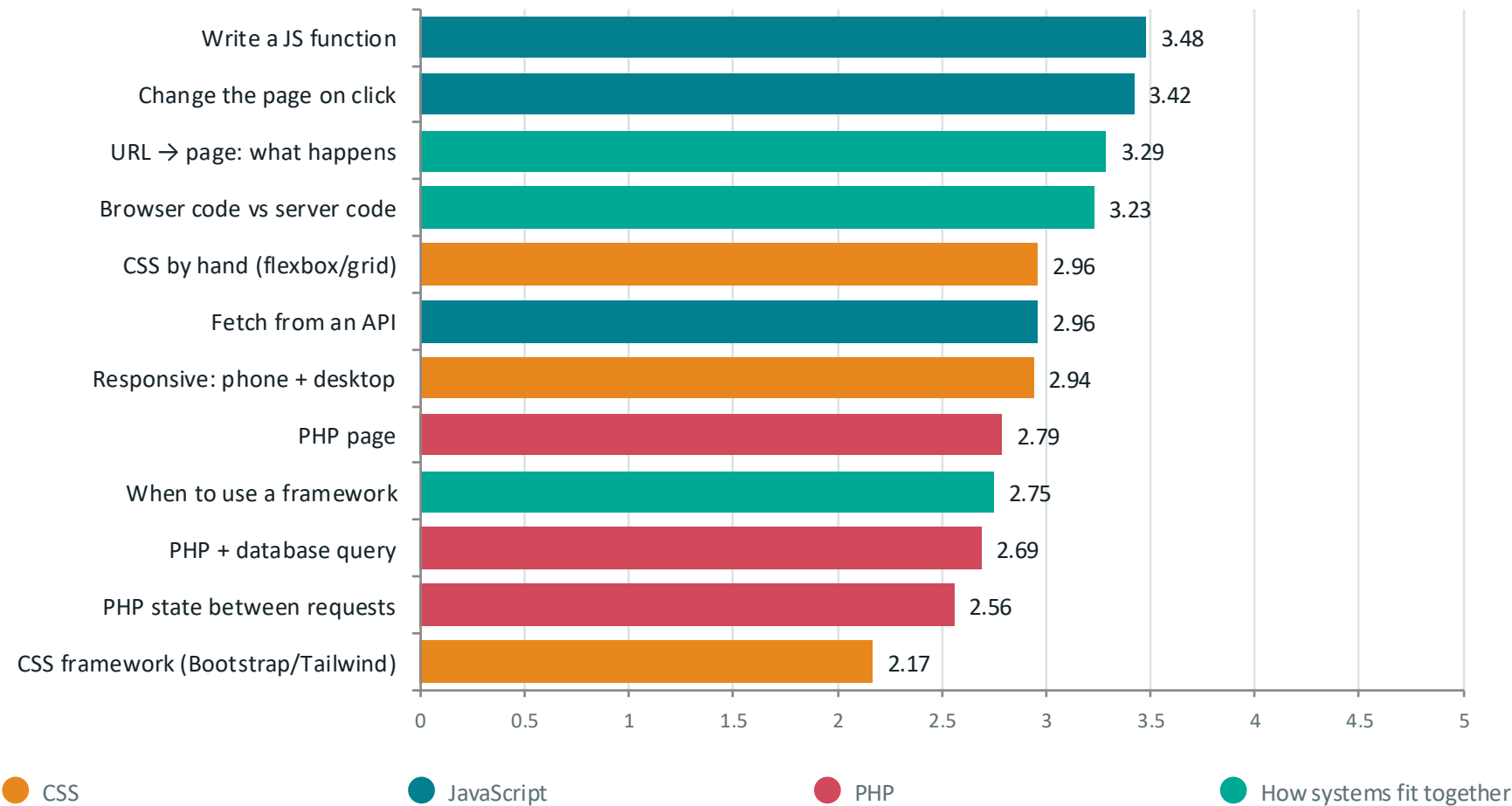
group average, 1–5

Also the widest spread. On browser-vs-server, answers ran the full 1–5.

This is where the class is least alike.

ALL TWELVE, RANKED

Strongest to weakest



Nothing clears 3.5

Not one of the twelve reached “could do it from memory” on average.

The floor is a framework

CSS frameworks sit almost a full point below everything else.

PHP clusters

All three PHP questions land within 0.23 of each other.

Opportunity to Review While Learning New Things

- Ask question related to other things that aren't clear.

What does a Docker
Container Do?

What problem does Docker
Solve?

Chapter 1 Intro

Docker: Origins and the Promise

- Unveiled by Solomon Hykes (dotCloud) in a 5-minute lightning talk at PyCon — March 15, 2013; open-sourced on GitHub within weeks
- Within a year, nearly the whole industry had heard of Docker — a new way to build, deliver, and run software
- Not virtualization: the first widely accessible tool built on **containerization** — Linux containers had existed since 2008, but Docker made them approachable
- Reshaped CI/CD expectations: fully automated pipelines instead of specialist-managed steps
- Reduces the communication burden between dev and ops teams — an isolation layer that lets each move faster
- **Atomic, throwaway containers** enforce best practices: immutable, portable deployment artifacts → more scalable, more reliable applications

What Docker Brings

Benefits of the Docker Workflow

- One standard packaging format (OCI images) — no separate build/release specialists needed
- App + OS dependencies bundled into a single image: the environment is always what you expect
- The exact same artifact is tested and shipped to every environment — no repackaging
- Abstracts apps from hardware without hypervisor overhead: containers are just processes on the shared kernel

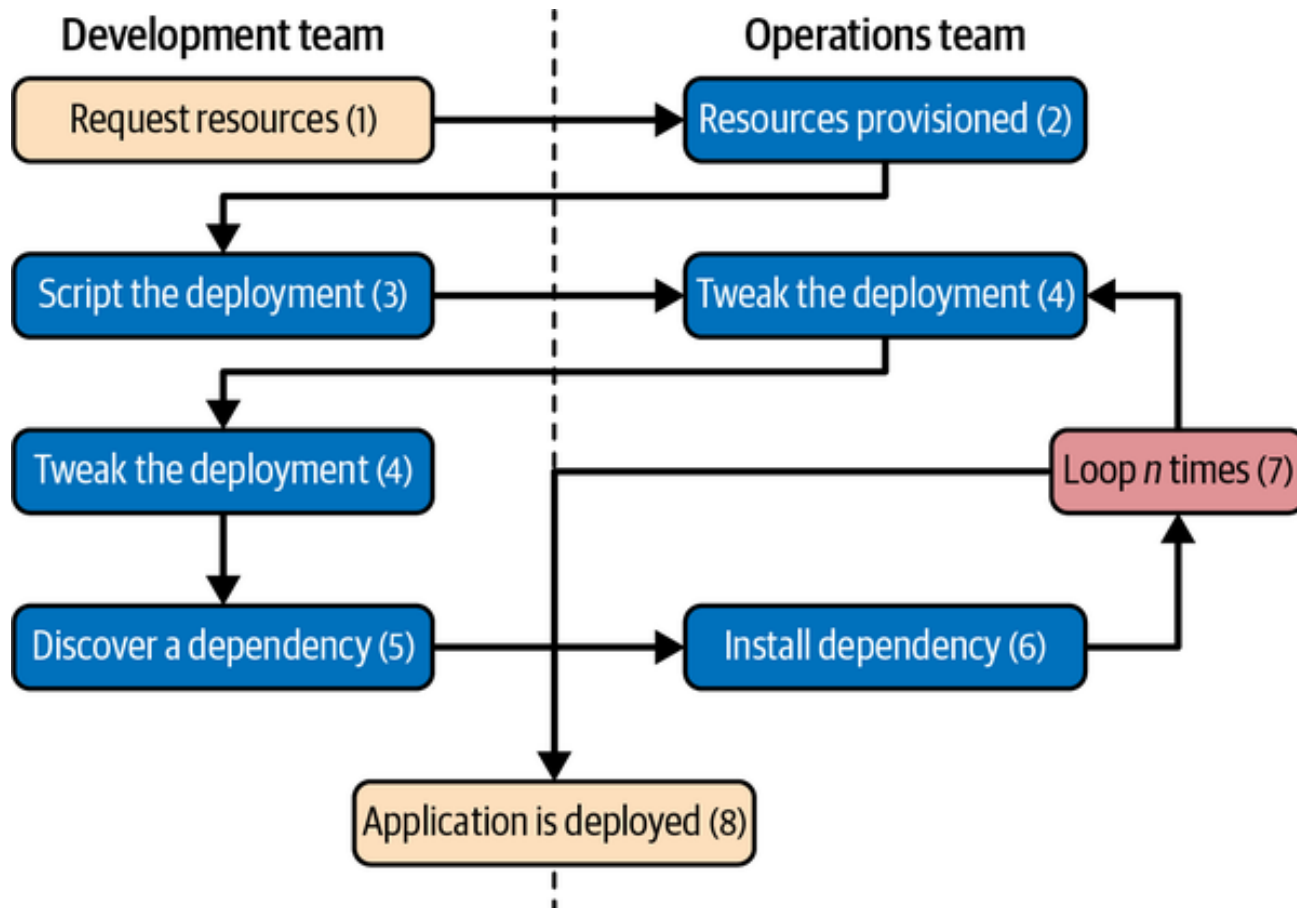
What Docker Isn't

What Docker Isn't (it complements, not replaces)

- Virtualization platform (VMware, KVM) — containers share the host kernel, no guest OS
- Cloud platform (OpenStack) — no provisioning of hosts, storage, or networks
- Config management (Puppet, Chef) — no ongoing state management
- Deployment framework (Capistrano) or dev environment (Vagrant)
- Workload orchestrator — that's Kubernetes, Swarm, Mesos

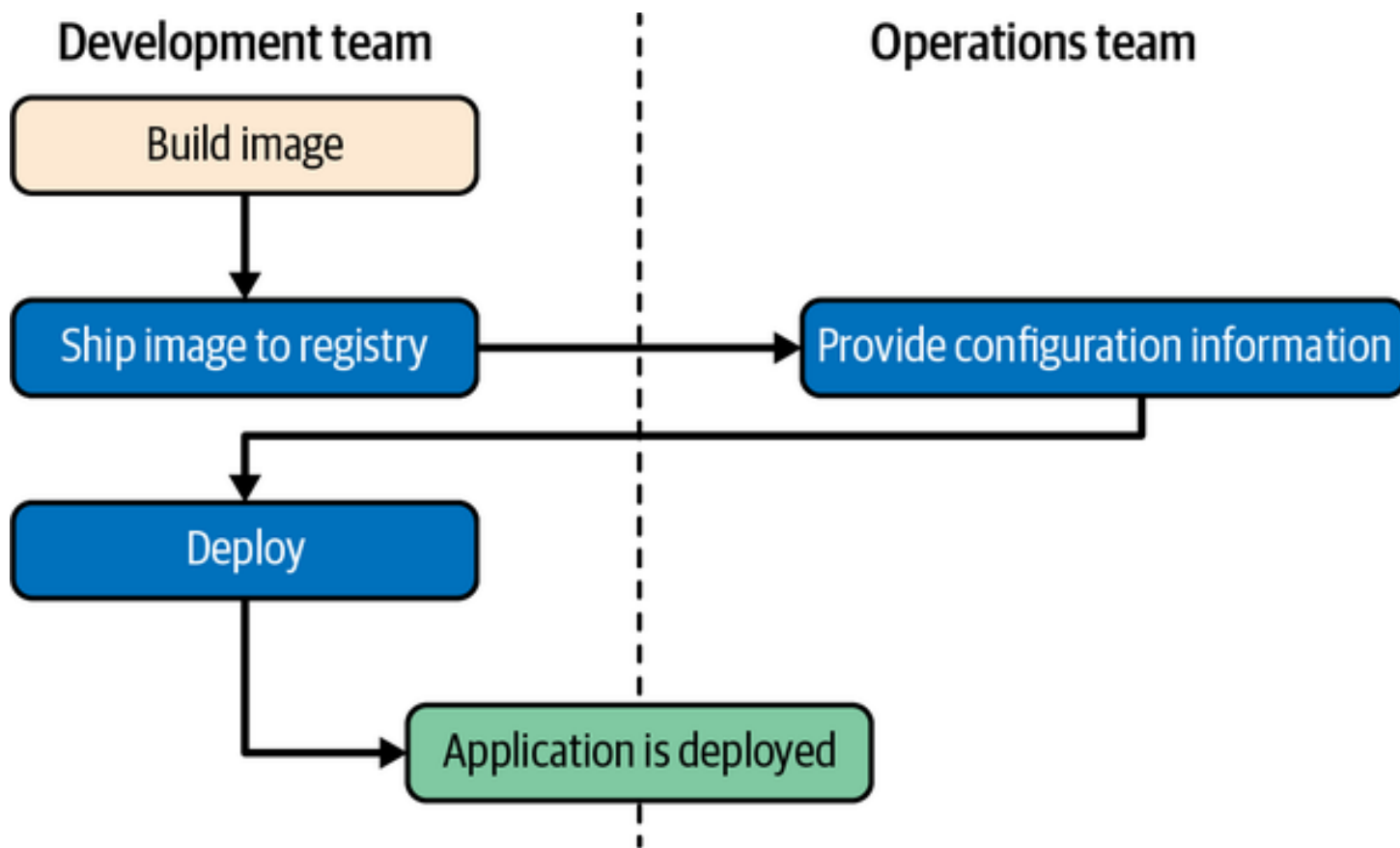
Chapter 2 The Docker Landscape

Before Docker



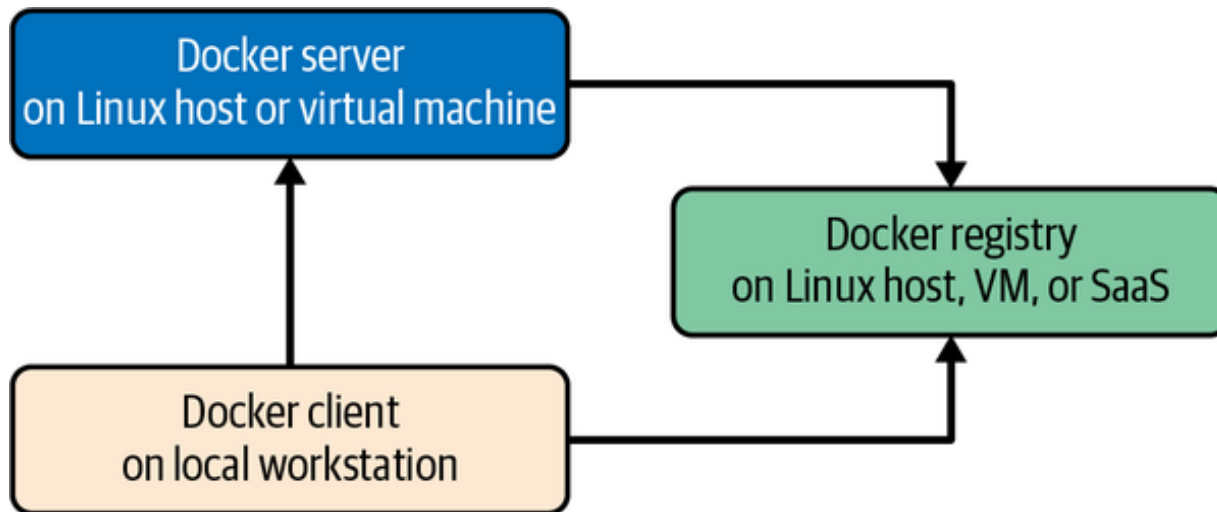
ISSUES

- Each new dependency can break deployment when the target environment differs from development.
- Infrastructure reflects years of undocumented administrator actions, manual changes, and one-off fixes.
- Repeated handoffs between development and operations create long feedback loops and rework.
- Deployment scripts become environment-specific, fragile, and difficult to reproduce or scale.



What Docker changes

- Bundles the application, runtime, libraries, and dependencies into a versioned image.
- Promotes the same image across development, testing, and production, reducing environment drift while configuration remains external.
- Dockerfiles make build instructions explicit, repeatable, and version-controlled.
- A registry handoff lets developers publish approved images while operations configures, deploys, and rolls them back predictably.



How to read this model

- The docker CLI is the client; dockerd exposes the Docker API and runs on the host.
- Behind dockerd, containerd manages container lifecycles and runc starts containers. docker-proxy and docker-init are used when needed.
- That complexity is hidden behind one API: one daemon can manage many containers on a host.
- The CLI can use a local socket or a secured remote connection. dockerd pulls and pushes images through a registry.

DOCKER DESKTOP BUNDLES THE DOCKER CLIENT AND A LOCAL DOCKER ENGINE

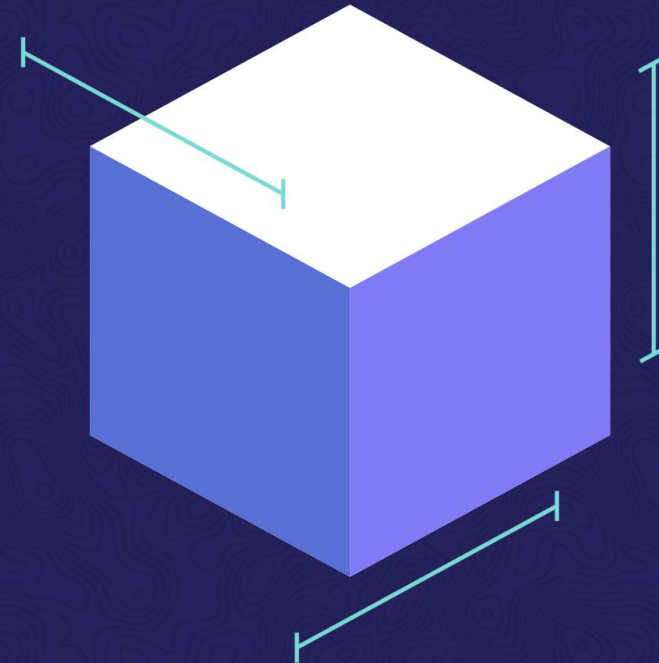
Containers Have Standards

Open Container Initiative

The **Open Container Initiative** is an open governance structure for the express purpose of creating open industry standards around container formats and runtimes.

Established in June 2015 by Docker and other leaders in the container industry, the OCI currently contains three specifications: the Runtime Specification (runtime-spec), the Image Specification (image-spec) and the Distribution Specification (distribution-spec). The Runtime Specification outlines how to run a “filesystem bundle” that is unpacked on disk. At a high-level an OCI implementation would download an OCI Image then unpack that image into an OCI Runtime filesystem bundle. At this point the OCI Runtime Bundle would be run by an OCI Runtime.

Learn more ↗



<https://opencontainers.org>

What is the Docker Business
Model?

Docker sells the developer experience around a free engine

- The core technology is free and open source — Docker Engine, containerd, and runc are openly licensed.
- Docker monetizes the tooling and services around that engine through tiered subscriptions.

REVENUE STREAMS

Docker Desktop

Subscription seats (Pro, Team, Business). Free for personal use; a paid license is required for larger companies.

Docker Hub

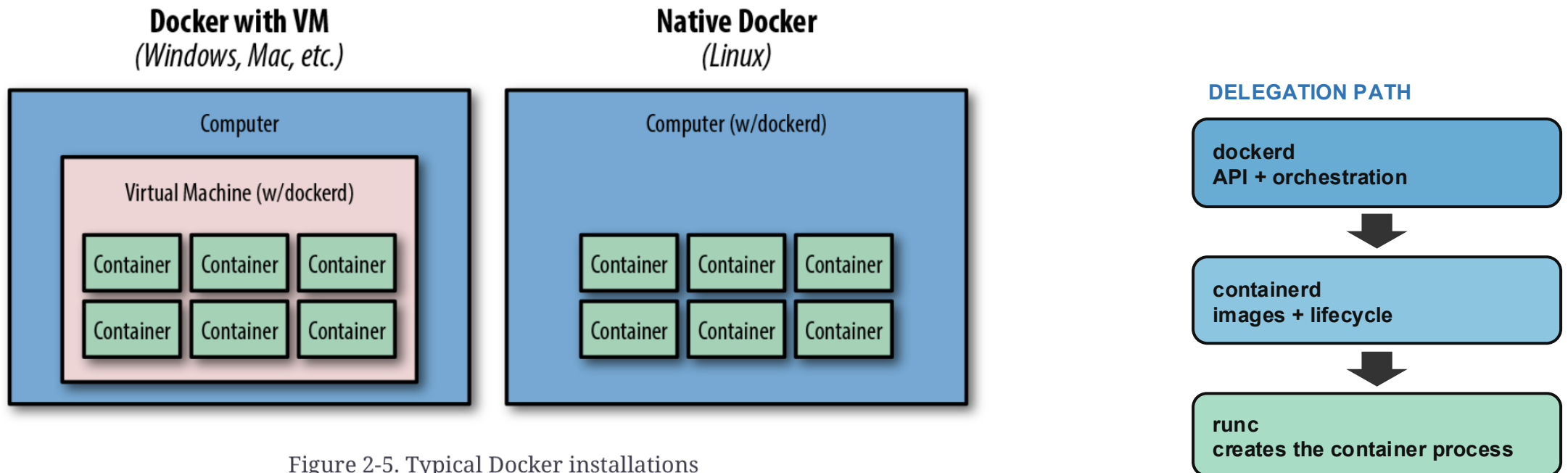
SaaS image registry. Paid plans add private repositories, higher image-pull limits, and automated builds.

Enterprise add-ons

Business tier: SSO, access controls, hardened images — plus products like Docker Scout and Build Cloud.

dockerd delegates execution to containerd

- dockerd exposes the Docker API and coordinates client requests.
- dockerd delegates image and container lifecycle tasks to containerd, which invokes runc to create the isolated container process.



Containers improve efficiency, consistency, and deployment speed

- **Process Isolation, Shared Kernel.** Namespaces and cgroups isolate workloads while avoiding the overhead of a separate guest OS.
- **Lightweight and Fast.** Containers can start quickly and allow greater workload density than virtual machines.
- **Immutable Deployments.** Rebuild and replace containers instead of patching running instances in place.
- **Portable, Repeatable Environments.** Versioned images package the application and its dependencies for consistent execution across hosts.
- **Externalized Configuration and State.** Inject configuration at runtime and keep persistent data in volumes or external services.

The Docker Workflow

- Revision Control

- Filesystem layers and image tags give specific versions to each.

- Build

- docker build turns the Dockerfile into a layered image; cached layers make rebuilds fast

- Test

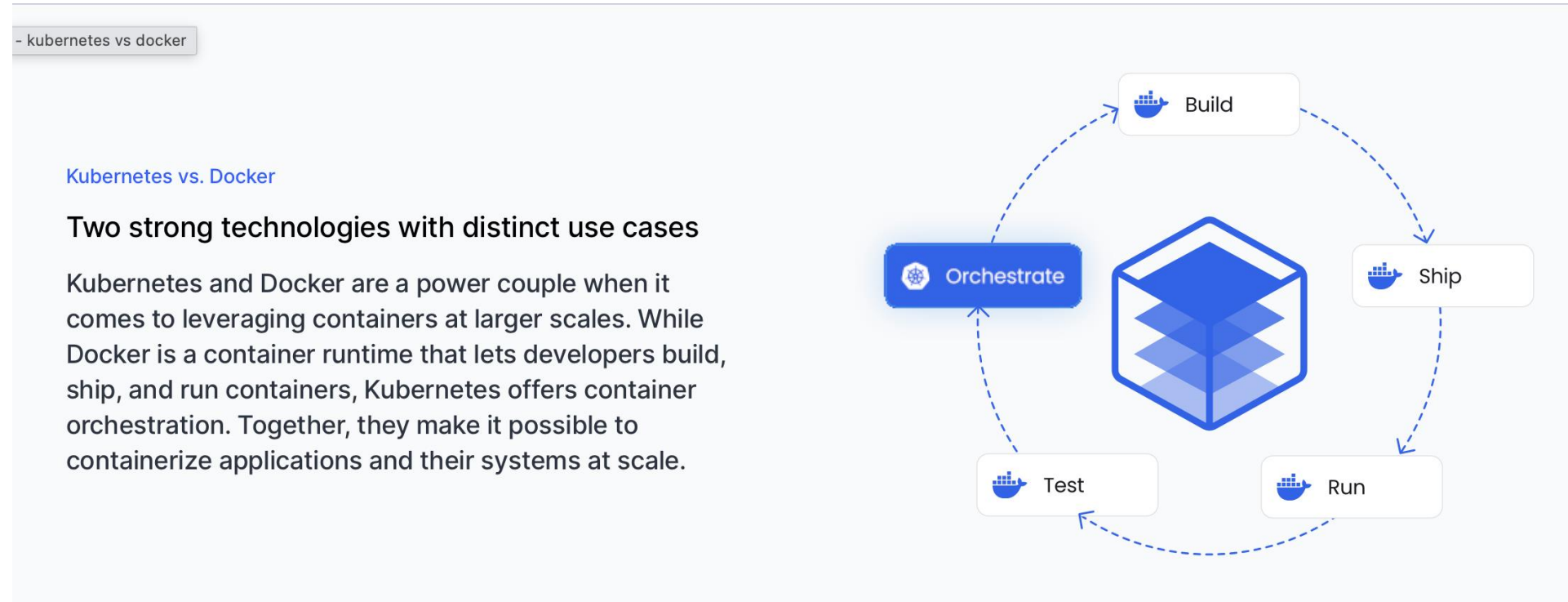
- Run the same image locally or in CI — identical environment eliminates “works on my machine”

- Package

- Tag the image and push it to a registry, ready to pull and run on any host

Docker builds images; Kubernetes runs containers at scale

They are complementary



“Fully automatic schedulers like [Kubernetes](#) or [Apache Mesos](#) with the [Marathon scheduler](#) are more powerful options that take nearly complete control of a pool of hosts on your behalf. Other commercial entries are widely available, such as [HashiCorp’s Nomad](#), [Mesosphere’s DC/OS \(Datacenter Operating System\)](#), and [Rancher](#).² The ecosystems of both free and commercial options continue to grow rapidly.”

Group Exercise

Containers

Container Hunt: Find Where Containers Are in the Wild

Each person contributes one unique, verifiable reference

1 SEARCH GITHUB

Look for a concrete container clue:

Dockerfile or Containerfile
compose.yaml
Kubernetes or Helm manifests
CI workflows that build or push images

2 SEARCH CLOUD INFRASTRUCTURE

Look for public or authorized evidence:

ECS / Fargate / EKS
Cloud Run / GKE
AKS / Container Apps
Registries, IaC, task or deployment definitions

3 DOCUMENT IT

Add the link, visible evidence, the container signal, and why containers are used.

SUCCESS

**Unique + credible +
explainable in 30 seconds**

Container in the Wild

Duplicate this slide once per contributor

CONTRIBUTOR [Name]	SOURCE [GitHub / cloud / other]
EVIDENCE [Paste a screenshot, code snippet, manifest excerpt, or architecture detail here] Crop to the exact clue. Do not expose credentials, secrets, or private infrastructure details.	
REFERENCE URL [Paste link] CONTAINER SIGNAL [Dockerfile, image, registry, orchestration, etc.] WHERE DOES IT RUN? [Local, CI, managed cloud, cluster, unknown] WHY CONTAINERS HERE? [One concise reason tied to the evidence]	
30-SECOND TAKEAWAY [This example shows that containers are used to...]	

Shared PPT

https://docs.google.com/presentation/d/1PEbrO70PhS61m8-t_AS8RsHtaLZQsO3H/edit?usp=sharing&oid=102358270837930135419&rtpof=true&sd=true

Section 2

https://docs.google.com/presentation/d/1fn8zkJZ39ZhOVPuBHU-UEwdLLPJ_n6N9/edit?usp=sharing&oid=102358270837930135419&rtpof=true&sd=true

Chapter 3 Installing

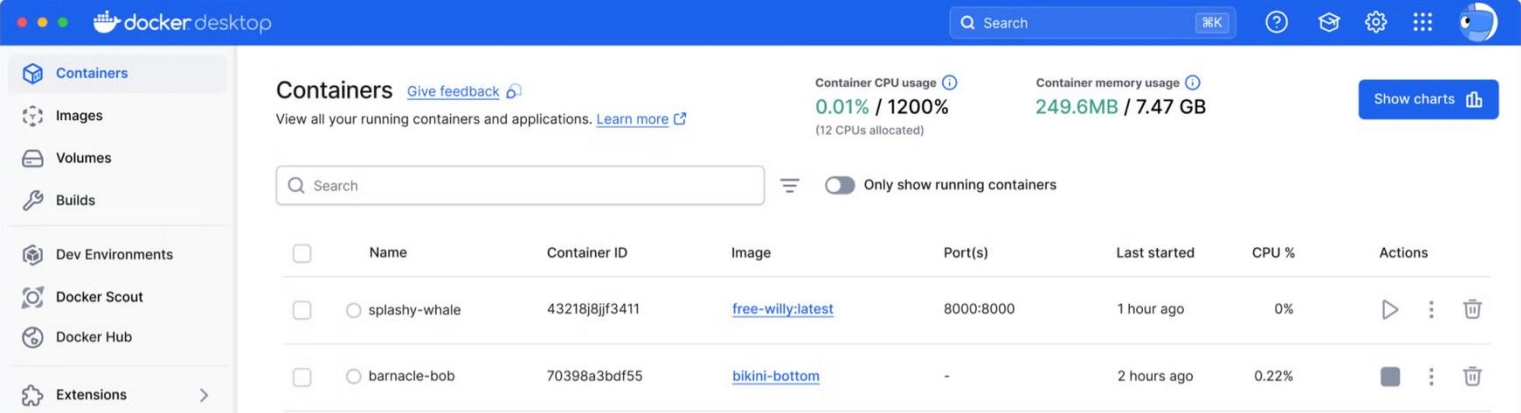
Docker Desktop

Docker Desktop

The #1 containerization software for developers and teams

Streamline development with Docker Desktop's powerful container tools.

[Download Docker Desktop](#) [Choose plan](#)



The screenshot shows the Docker Desktop application window. On the left is a sidebar with navigation links: Containers, Images, Volumes, Builds, Dev Environments, Docker Scout, Docker Hub, and Extensions. The main area is titled 'Containers' and displays system usage statistics: 'Container CPU usage 0.01% / 1200% (12 CPUs allocated)' and 'Container memory usage 249.6MB / 7.47 GB'. Below these is a search bar and a toggle for 'Only show running containers'. A table lists the running containers:

☐	Name	Container ID	Image	Port(s)	Last started	CPU %	Actions
☐	splashy-whale	43218j8jff3411	free-willy:latest	8000:8000	1 hour ago	0%	
☐	barnacle-bob	70398a3bdf55	bikini-bottom	-	2 hours ago	0.22%	

To support the growing demand for Docker tooling in development environments, Docker has released easy-to-use implementations for macOS and Windows. These appear to run natively but are still utilizing a small Linux virtual machine to provide the Docker server and Linux kernel.

<https://www.docker.com/products/docker-desktop/>

Chapter 4 Docker Images

A Dockerfile is the recipe for building an image

Docker reads the instructions from top to bottom and creates a reusable image layer by layer.

- **FROM** Selects the base image and starting filesystem.
- **LABEL** Adds metadata to the image as key-value pairs.
- **ARG / ENV** Defines build-time arguments and runtime environment variables.
- **RUN** Executes commands during the image build, such as installing packages.
- **COPY** Adds application files and configuration into the image.
- **WORKDIR** Sets the default directory for later instructions and commands.
- **CMD** Provides the default process that runs when a container starts.

```
FROM node:18.13.0
```

```
ARG email="anna@example.com"
```

```
LABEL "maintainer"=$email
```

```
LABEL "rating"="Five Stars" "class"="First Class"
```

```
USER root
```

```
ENV AP /data/app
```

```
ENV SCPATH /etc/supervisor/conf.d
```

```
RUN apt-get -y update
```

```
# The daemons
```

```
RUN apt-get -y install supervisor
```

```
RUN mkdir -p /var/log/supervisor
```

```
# Supervisor Configuration
```

```
COPY ./supervisord/conf.d/* $SCPATH/
```

```
# Application Code
```

```
COPY *.js* $AP/
```

```
WORKDIR $AP
```

```
RUN npm install
```

```
CMD ["supervisord", "-n"]
```



mysql

Docker Official Image · 1B+ · 10K+

MySQL is a widely used, open-source relational database management system (RDBMS).

DATABASES & STORAGE

Overview

Tags

Quick reference

- Maintained by:
[the Docker Community and the MySQL Team](#)
- Where to get help:
[the Docker Community Slack](#), [Server Fault](#), [Unix & Linux](#), or [Stack Overflow](#)

Supported tags and respective Dockerfile links

- `26.7.0`, `26.7`, `26`, `innovation`, `latest`, `26.7.0-oraclelinux9`, `26.7-oraclelinux9`, `26-oraclelinux9`, `innovation-oraclelinux9`, `oraclelinux9`, `26.7.0-oracle`, `26.7-oracle`, `26-oracle`, `innovation-oracle`, `oracle`

Linked to version control

- Each tag on Docker Hub links to the exact Dockerfile it was built from
- Dockerfiles live in a Git repo — tags map to branches and commits
- Automated builds: a push to the repo triggers a rebuild of the image
- Full traceability: running container → image tag → Dockerfile → commit

WARNING

In many examples online and in this book, you will see people use the `latest` tag for a container image. This is useful when you're getting started and when you're writing examples, as it will always grab the most recent build of an image. But since this is a floating tag, it is a really bad idea to use `latest` in most production workflows, as your dependencies can get updated out from under you, and it is impossible to roll back to `latest` because the old version is no longer the one tagged `latest`. It also makes it hard to verify if the same image is running on different servers. The rule of thumb is: don't use the `latest` tag in production. It's not even a good idea to use the `latest` tag from upstream images, for the same reasons.

https://hub.docker.com/_/mysql

Two isolated machines, then a system

TUESDAY



no connection

FRIDAY



one network · DNS by name · no published db port

MySQL, with no Dockerfile at all

A container's filesystem dies with the container.

WITHOUT A VOLUME

```
$ docker rm -f db1
$ docker run --name db1 ... -d mysql:8.4
$ docker exec db1 mysql -e "SELECT * FROM students"
```

```
ERROR 1146 (42S02):
  Table 'app.students' doesn't exist
```

WITH `-v db1_data:/var/lib/mysql`

```
$ docker rm -f db1
$ docker run --name db1 \
  -v db1_data:/var/lib/mysql -d mysql:8.4
```

```
id  name
1   Ada Lovelace
```

Also worth doing: get a bash shell inside it. PID 1 is mysqld -- the database IS the machine.

Build the schema into the image

Scripts in `/docker-entrypoint-initdb.d` run once — only on an empty data directory.

CHANGED `init.sql`, REBUILT, RESTARTED

```
$ docker build -t itws-db:1 .  
$ docker rm -f db2 && docker run ... -v db2_data  
$ ... -e "SELECT name FROM students"
```

```
Ada Lovelace  
Grace Hopper  
Alan Turing          <- 3 rows. No change.
```

AFTER `docker volume rm db2_data`

```
$ docker volume rm db2_data  
$ docker run ... -v db2_data:/var/lib/mysql
```

```
Ada Lovelace  
Grace Hopper  
Alan Turing  
Katherine Johnson    <- there it is
```

Nine out of ten "my schema change isn't showing up" problems all semester are this.

Apache, and the filesystem

A bind mount is not a copy. It is one set of files, visible from two places.

EDIT ON THE LAPTOP -> SERVED INSTANTLY

```
$ docker run -d -p 8080:80 \
  -v "$PWD/site":/usr/local/apache2/htdocs \
  httpd:2.4

# edit site/index.html, save, reload
<h1>EDITED ON HOST</h1>
# no rebuild. no restart. no docker command.
```

WRITE INSIDE -> APPEARS ON THE LAPTOP

```
$ docker exec -it web1 bash
# echo hi > /usr/local/apache2/htdocs/hello.html
# exit
$ ls site/

hello.html    index.html    style.css
```

Source code in a bind mount. Database files in a named volume. Mixing those up is a real bug.

PHP writing to disk — still no database

The same rule, now in the web tier. Restart is not remove.

`docker restart gb`

```
# sign the guestbook twice, then:
```

```
$ docker restart gb
```

```
2 entries
```

```
# the process stopped and started.
```

```
# the filesystem was never touched.
```

`docker rm -f gb && docker run ...`

```
$ docker rm -f gb
```

```
$ docker run -d -p 8080:80 ex4-gb
```

```
0 entries
```

```
# same image, same command,
```

```
# new container, empty guestbook.
```

Nothing about "a web server" makes it different from "a database". The rule is about containers.

Before you go

- Answer questions from in-class/wk2/starter/ANSWERS-tue.md
- Push them to your repo.
- See me if you didn't get an invite.